

Inter Process Communication (IPC) - Fundamentals and Shared Memory

Table of Contents

1. [What is a Process?](#)
 2. [What is Inter Process Communication?](#)
 3. [Why Do We Need IPC?](#)
 4. [Independent vs. Cooperating Processes](#)
 5. [The Two Fundamental IPC Models](#)
 6. [Shared Memory in Detail](#)
 7. [Shared Memory System Calls](#)
 8. [Working Code Example](#)
 9. [The Producer-Consumer Problem](#)
 10. [Synchronization Problems](#)
 11. [Advantages and Disadvantages](#)
 12. [Real-World Use Cases](#)
 13. [Summary](#)
 14. [References](#)
-

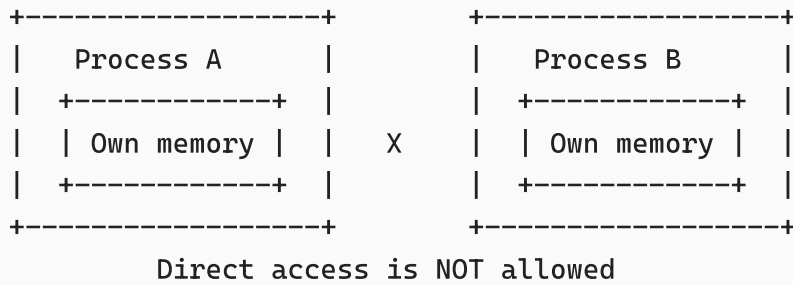
1. What is a Process?

A **process** is a program that is currently running. When you double-click a browser or run `./a.out`, the operating system (OS) creates a process for it.

Every process gets its own:

- **Address space**: its own private area of memory (code, data, heap, stack)
- **Process ID (PID)**: a unique number that identifies it
- **Resources**: open files, CPU registers, and so on

The most important point for this topic: **the OS keeps processes isolated from each other**. One process cannot directly read or write another process's memory. This is good for safety, because a bug in one program will not corrupt another. But it creates a problem: what if two processes *need* to share data?



2. What is Inter Process Communication?

Inter Process Communication (IPC) is the set of mechanisms an operating system provides so that separate processes can **exchange data** and **coordinate their actions**.

Since processes cannot touch each other's memory directly, the OS acts as a middleman and offers safe "channels" for communication.

Simple analogy: Think of two people working in separate, locked rooms. They cannot walk into each other's rooms (process isolation). IPC is like a shared whiteboard between the rooms (shared memory), or a mail slot in the wall (message passing).

3. Why Do We Need IPC?

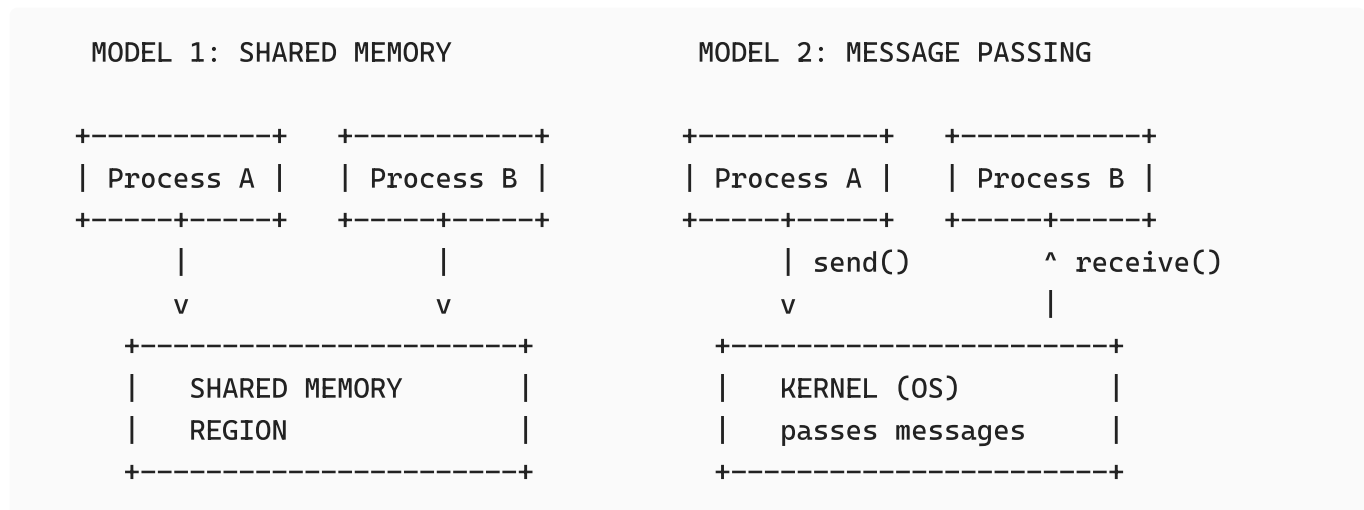
Reason	Explanation	Example
Information sharing	Several processes need the same data	Multiple programs reading one shared clipboard
Computation speed-up	Split a big task into parts that run in parallel	A video encoder using several processes for different frames
Modularity	Break a big system into smaller, separate programs	A web server, database, and cache running as different processes
Convenience	A user runs many tasks at once that need to cooperate	Copying text from a browser and pasting into an editor
Synchronization	Processes must wait for or signal each other	A process waits until another finishes writing a file

4. Independent vs. Cooperating Processes

- **Independent process:** does not affect and is not affected by any other process. It shares no data with anyone.
- **Cooperating process:** can affect or be affected by other processes. It **needs IPC** because it shares data or works together with others.

5. The Two Fundamental IPC Models

Almost every IPC mechanism falls into one of two models.



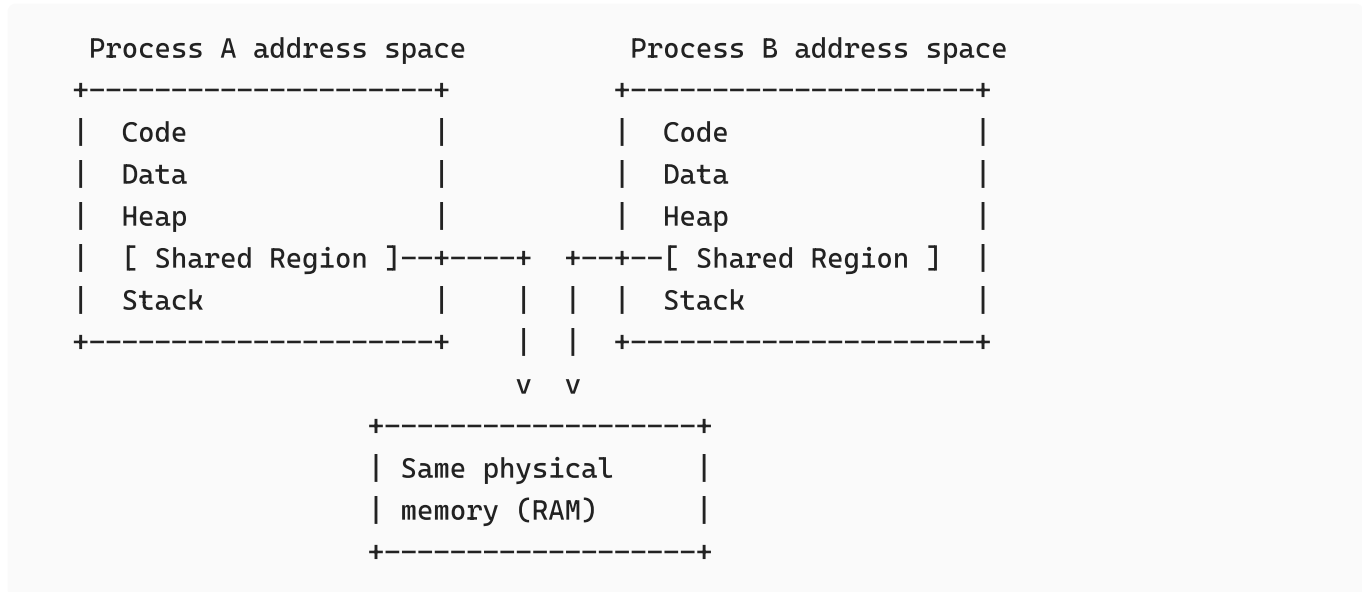
Feature	Shared Memory	Message Passing
How it works	A memory region is mapped into multiple processes	Processes send and receive messages through the kernel
Speed	Very fast (no kernel involvement after setup)	Slower (every message goes through the kernel)
Kernel involvement	Only at setup	For every send and receive
Synchronization	Programmer must handle it	Handled largely by the OS
Best for	Large amounts of data	Small amounts of data, distributed systems
Ease of use	Harder to get right	Easier

This document focuses on **Shared Memory**. Message passing is covered in the message queues document.

6. Shared Memory in Detail

6.1 The Idea

With shared memory, the OS sets up **one region of memory that two or more processes can both access**. After this setup, processes read and write to that region just like ordinary memory, with no system calls needed for each data transfer.



Both processes map the **same physical memory** into their own address spaces. If A writes something into it, B can see it immediately.

6.2 Why is it the Fastest IPC Method?

In message passing, every piece of data is **copied** from the sender to the kernel and then from the kernel to the receiver. In shared memory, **no copying** happens. The data just sits there, and both processes can see it.

6.3 The Catch

Because the OS does not manage access to the shared region, **the programmers must make sure two processes do not modify the same data at the same time**. That is what synchronization is about (Section 10).

6.4 Steps to Use Shared Memory

1. **Create** a shared memory object (one process does this).
2. **Set its size**.
3. **Attach / map** it into each process's address space.
4. **Read and write** using normal pointer operations.

5. **Detach / unmap** when finished.
 6. **Remove** the shared memory object (so it does not stay in the system).
-

7. Shared Memory System Calls

There are two families of APIs on Linux: **POSIX** (modern, simpler, used here) and **System V** (older).

POSIX Shared Memory

Function	Purpose
<code>shm_open(name, flags, mode)</code>	Create or open a named shared memory object. Returns a file descriptor.
<code>ftruncate(fd, size)</code>	Set the size of the shared memory object.
<code>mmap(addr, size, prot, flags, fd, offset)</code>	Map the object into the process's address space. Returns a pointer.
<code>munmap(ptr, size)</code>	Unmap it from the address space.
<code>close(fd)</code>	Close the file descriptor.
<code>shm_unlink(name)</code>	Delete the shared memory object.

System V Shared Memory (for reference)

Function	Purpose
<code>shmget()</code>	Create or get a shared memory segment
<code>shmat()</code>	Attach the segment to the process
<code>shmdt()</code>	Detach the segment
<code>shmctl()</code>	Control operations, such as removing the segment

8. Working Code Example

The child process writes a message into shared memory and the parent reads it. The parent calls `wait()` so it only reads after the child has finished writing. This is a simple form of synchronization.

File: code/shared_memory/shm_demo.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>           // O_CREAT, O_RDWR
#include <sys/mman.h>        // shm_open, mmap, munmap, shm_unlink
#include <sys/stat.h>        // mode constants
#include <sys/wait.h>        // wait
#include <unistd.h>          // fork, ftruncate, close

#define SHM_NAME "/ipc_demo_shm"
#define SHM_SIZE 4096

int main(void) {
    // 1. Create the shared memory object
    int fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666);
    if (fd == -1) {
        perror("shm_open");
        exit(1);
    }

    // 2. Set its size
    if (ftruncate(fd, SHM_SIZE) == -1) {
        perror("ftruncate");
        exit(1);
    }

    // 3. Map it into this process's address space
    char *ptr = mmap(NULL, SHM_SIZE, PROT_READ | PROT_WRITE,
                     MAP_SHARED, fd, 0);
    if (ptr == MAP_FAILED) {
        perror("mmap");
        exit(1);
    }

    pid_t pid = fork();

    if (pid == 0) {
        // ---- CHILD: writer ----
        printf("[Child ] Writing to shared memory...\n");
        strcpy(ptr, "Hello from the child process!");
        munmap(ptr, SHM_SIZE);
        exit(0);
    } else {
        // ---- PARENT: reader ----
    }
}
```

```

    wait(NULL); // wait until child has finished writing
    printf("[Parent] Read from shared memory: %s\n", ptr);

    // 5 and 6. Clean up
    munmap(ptr, SHM_SIZE);
    close(fd);
    shm_unlink(SHM_NAME);
}
return 0;
}

```

Compile and run:

```

gcc shm_demo.c -o shm_demo -lrt
./shm_demo

```

(`-lrt` is needed on older Linux systems and is harmless on newer ones.)

Sample output:

```

[Child ] Writing to shared memory...
[Parent] Read from shared memory: Hello from the child process!

```

How it works: `fork()` creates a child that inherits the mapping. Because we used `MAP_SHARED`, the child's write is visible to the parent. With `MAP_PRIVATE` it would not be.

9. The Producer-Consumer Problem

This is the classic problem used to explain cooperating processes and shared memory.

- The **producer** creates data and puts it into a **buffer** (shared memory).
- The **consumer** takes data out of the buffer and uses it.



Rules that must hold:

1. The producer must **not add** data if the buffer is **full**.

2. The consumer must **not remove** data if the buffer is **empty**.
3. The producer and consumer must **not access the buffer at the exact same moment**.

Two types of buffers:

Type	Meaning
Unbounded buffer	No limit on size. The consumer may wait, but the producer never waits.
Bounded buffer	Fixed size. The producer waits when full and the consumer waits when empty.

Bounded buffers are usually implemented as a **circular buffer** with two indices, `in` (next free slot) and `out` (next full slot):

```
#define BUFFER_SIZE 10

typedef struct {
    int data[BUFFER_SIZE];
    int in;    // where the producer writes next
    int out;   // where the consumer reads next
} Buffer;

// Producer (simplified, without synchronization):
while (((in + 1) % BUFFER_SIZE) == out)
    ;    // buffer full: wait
buffer[in] = item;
in = (in + 1) % BUFFER_SIZE;

// Consumer (simplified, without synchronization):
while (in == out)
    ;    // buffer empty: wait
item = buffer[out];
out = (out + 1) % BUFFER_SIZE;
```

This simple version keeps one slot empty to tell "full" apart from "empty". A full production-ready version would use semaphores (covered in Rafath's document).

10. Synchronization Problems

10.1 Race Condition

A **race condition** happens when two or more processes access shared data at the same time, and the final result depends on **who runs first**.

Example: Two processes both run `counter++` on a shared counter that starts at 5. Internally, `counter++` is three steps:

1. LOAD `counter` into register
2. ADD `1` to register
3. STORE register back into `counter`

If the processes interleave:

```
Process A: LOAD  counter    (reads 5)
Process B: LOAD  counter    (reads 5)
Process A: ADD   -> 6
Process B: ADD   -> 6
Process A: STORE 6
Process B: STORE 6
```

Result: `counter = 6` (expected: 7)

One update is lost. This kind of bug is hard to find because it happens only occasionally.

10.2 Critical Section

The part of the code that accesses shared data is called the **critical section**. A correct solution must satisfy three requirements:

Requirement	Meaning
Mutual exclusion	Only one process can be in its critical section at a time
Progress	If no one is in the critical section, a waiting process must be allowed in without unnecessary delay
Bounded waiting	A process cannot be made to wait forever

```
do {
    [ ENTRY SECTION ]    <- request permission (lock / wait)
    CRITICAL SECTION    <- access shared memory
    [ EXIT SECTION ]     <- release permission (unlock / signal)
```

```
} while (true);
```

10.3 Tools to Solve It

Tool	Description
Mutex (lock)	A lock that only one process/thread can hold at a time
Semaphore	A counter-based signal used to control access to resources
Monitor	A high-level construct that bundles data with the operations that protect it

Semaphores and mutexes are explained in Rafath's document.

11. Advantages and Disadvantages

Advantages

- **Fastest** IPC method, since there is no copying of data through the kernel
- Efficient for **large amounts of data**
- The kernel is needed only during setup
- Data stays available for as long as the segment exists

Disadvantages

- **No built-in synchronization**, so race conditions are the programmer's responsibility
 - Works only for processes on the **same machine**
 - Harder to debug
 - Shared memory objects can **linger in the system** if not cleaned up (check with `ls /dev/shm`)
 - Poor use can create security risks if permissions are too open (for example `0666`)
-

12. Real-World Use Cases

Use Case	How Shared Memory Helps
Databases (PostgreSQL)	Multiple server processes share a buffer cache

Use Case	How Shared Memory Helps
Web browsers (Chrome)	Renderer processes share graphics data with the main process
Video and audio processing	Large frames are shared without copying
Scientific computing	Parallel programs share big arrays
Multiplayer game servers	Shared game state across processes

13. Summary

- Processes are isolated, so the OS provides **IPC** for them to communicate.
 - There are two models: **shared memory** and **message passing**.
 - **Shared memory** maps the same physical memory into several processes. It is the fastest method but needs **manual synchronization**.
 - The **producer-consumer problem** is the standard example.
 - Unsynchronized access causes **race conditions**, and the **critical section** must be protected with mutexes or semaphores.
-

14. References

1. Silberschatz, Galvin, Gagne. *Operating System Concepts* (Chapter 3: Processes, Chapter 6: Synchronization).
2. Linux manual pages: `man 7 shm_overview`, `man 3 shm_open`, `man 2 mmap`.
3. Kerrisk, M. *The Linux Programming Interface*. No Starch Press.
4. GeeksforGeeks: Inter Process Communication.